

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.		
Student Name:	Murshidha.M	Reg. No.:	
Section / Batch:	A	Date:	26/08/2026

Code of Conduct

I Murshidha.M (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Murshidha

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

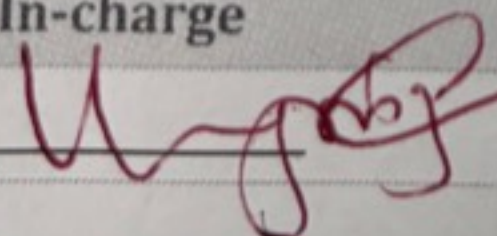
Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
GRAND TOTAL:			100 Marks

Annexure A – Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)
 - a) Clustering effect
 - b) Memory usage
 - c) Performance under high load factor
 - d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of: (20 Marks)
 - a) Clustering issues
 - b) Collision resolution efficiency
 - c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)
 - a) Practical significance
 - b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
 - a) Space complexity (stack usage)
 - b) Ease of implementation
 - c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CO4-ASSESSMENT
TOOL-3

Name: Murshidha.m
Reg No: 192571007
Code: CSA0305.

① Factor	Linear probing	Separate chaining
a) clustering	causes primary clustering because elements are stored in consecutive slots.	No primary clustering collided elements are stored in separate chain
(b) memory usage	uses less memory because elements are stored in hash table	uses more memory because extra linked-list nodes.
(c) high load factor	Performance decreases rapidly as the table becomes full.	performs better because multiple elements can be stored.
(d) Large-Scale Application	Suitable when memory efficiency and cache performance are important.	more Suitable Scale application because it handles collision and high load factor

②.

Factor	Quick Sort	Merge Sort
Time complexity	Average : $O(n \log n)$ Worst : $O(n^2)$	Best / Average / Worst : $O(n \log n)$
Memory Linked List	Usually $O(\log n)$ recursion stack	$O(1)$ extra if merging is done by changing links
Cache Behaviour	Poor for linked lists because nodes are scattered in memory	Also not ideal but sequential traversal makes it better suited
Linked List	Less suitable	Highly suitable
Stability	Usually not stable	Stable
Implementation for Linked List	partitioning is more difficult	Easy to split and merge using pointers.

* takes guaranteed $O(n \log n)$ time

* Does not require random access

• Avoids difficult partitioning required by quick sort.

	Quadratic Probing	Double Hashing
Issues	Avoids primary cluster But many suffer from secondary cluster	Avoids Both primary and secondary cluster
Collision Resolution Efficiency	Better than Linear Probing, but probe sequence may repeat for keys with same value.	More efficient because it uses a second hash function to generate probe.
Dense Hash Table.	Performance decrease as the table becomes very full	Better suited for dense tables because it distributes collision

Double hashing is generally better than Quadratic probing especially for dense hash table.
clustering and provides more efficient collision resolution.

Double hashing > Quadratic Probing > Linear probing for handling collision in dense.

4,

Factor	Best case complexity	Worst-case com
meaning	Minimum time an algorithm takes for favorable input	Maximum time an algorithm may take for unfavourable input
Practical Significance	Shows how fast the algorithm can be in ideal situation.	Shows the performance guarantee and possible maximum delay.
Example impact on Selection	Quick Sort: $O(n \log n)$ Useful when input are usually favorable	Quick Sort: $O(n^2)$ Important when predictable and reliable

(a) practical significance:

* Best case helps understand the algorithm's performance

* worst case helps identify possible performance.

(B) impact on Algorithm selection.

* If input conditions are predictable and favourable.

	Recursive Quick Sort	Interactive Quick Sort
Space Complexity	uses recursive stack: $O(\log n)$ average, $O(n)$ worst case	uses an explicit stack: $O(\log n)$ average, $O(n)$ worst case.
Ease of Implementation	Easier and simpler to write and understand.	more complex because stack management is done manual.
Stack Overflow risk	higher, especially with poor pivot selection	lower, since recursion is avoided memory is controlled.

Wong